CS 498 — Data Management in the Cloud

Assignment 3: Distributed NoSQL, Tunable Consistency & Python API

Design Document

Team CloudSync
Prathmesh Sanjay Mulay · Chaitanya Nirantar · Anisha Kango

1. Bigtable Design

1.1 Row Key and Sharding Strategy

The Bigtable row key is HASH(VIN)#VIN. With the use of the hash prefix, the records will be distributed among tablets rather than being stored within one lexicographical range. The VIN still resides in the row key after the separator for the purpose of uniqueness.

Sharding through hashing is the main distribution technique since it involves the process of inserting in large batches and does not rely on scanning of VIN ranges.

1.2 Column Families

Column Family	Example Columns / Purpose
vehicle	make, model, model_year, electric_vehicle_type
location	county, city, state, postal_code, vehicle_location, geoid_2020
technical	electric_range, electric_utility
administrative	cafv_eligibility, legislative_district, dol_vehicle_id

1.3 Hash Sharding vs. Range Sharding

Approach	Advantage	Trade-off
Hash sharding	Evenly distributes rows and writes; reduces hotspot risk.	Less natural for ordered/range queries.
Range sharding	Efficient for ordered/range queries because nearby keys remain together.	Sequential or skewed keys can create hotspots.

2. MongoDB Atlas Architecture

MongoDB Atlas has been configured to be a replica set comprising of three nodes. The database is ev_db while the collection name is vehicles. The total number of records in the final dataset is 300,142 vehicles. No backups have been taken for the assignment.

Network access is granted to the fixed VM IP 34.60.92.184 and to the course autograder IP 136.64.84.147.

3. Python API and Tunable Consistency

3.1 Write Concern

Endpoint	Write Concern	Purpose
/insert-fast	w=1	Acknowledged primary write optimized for lower latency.
/insert-safe	w='majority'	Acknowledged after replication to a majority of replica-set members.

3.2 Read Preference

Endpoint	Read Preference	Purpose
/count-tesla-primary	PRIMARY	Reads Tesla records from the primary.
/count-bmw-secondary	SECONDARY_PREFERRED	Uses a secondary when available and falls back to the primary.

3.3 Implementation

The Flask application exposes the required endpoints on port 8000. MongoClient reads MONGO_URI from an environment variable so credentials are not hard-coded.

```
fast_collection = vehicles.with_options(write_concern=WriteConcern(w=1))
safe_collection = vehicles.with_options(write_concern=WriteConcern(w='majority'))
primary_collection = vehicles.with_options(read_preference=ReadPreference.PRIMARY)
secondary_preferred_collection = vehicles.with_options(
    read_preference=ReadPreference.SECONDARY_PREFERRED
)
```

4. CAP / Consistency Scenario

The configuration of the replica set illustrates the relationship between the guarantees of consistency and availability in case of the failure of nodes or the network. In the case of the fast endpoint, since w=1, the primary is able to acknowledge the write request without the need for replicating the data among the majority. This results in low latency but allows a period where the recently inserted document does not exist in the majority of nodes yet.

For the safe endpoint, since w='majority', it waits for the acknowledgment by a majority of the replica set, which provides higher durability and consistency guarantees but comes with high latency.

5. Benchmark Results

A benchmark of 500 sequential public HTTP POST requests was executed for each write endpoint. Requests were sent through the public VM endpoint, so network and API overhead were included.

Metric	/insert-fast	/insert-safe	Difference
Requests	500	500	0
Average latency	114.77 ms	130.22 ms	+15.45 ms
Minimum latency	91.16 ms	94.05 ms	+2.89 ms
Maximum latency	941.58 ms	1165.80 ms	+224.22 ms

The majority-write endpoint averaged 130.22 ms versus 114.77 ms for the fast endpoint, an increase of approximately 13.46%. This is consistent with the additional replication acknowledgement required by majority write concern.

6. Validation

Test	Observed Result	Status
Public health endpoint	HTTP 200; status=ok	PASS
POST /insert-fast	HTTP 201; ID returned	PASS
POST /insert-safe	HTTP 201; ID returned	PASS
GET /count-tesla-primary	122,291	PASS
GET /count-bmw-secondary	12,395	PASS
Final collection count	300,142	PASS
Public VM endpoint	34.60.92.184:8000	PASS

7. Conclusion

The system architecture involves a Bigtable-based hash sharding system coupled with the replica-set architecture of the MongoDB Atlas and the Flask/PyMongo API with tunable read/write consistency explicitly shown. The benchmark shows the trade-off of low-latency acknowledged vs. majority acknowledged writes in terms of latency. The architecture has been publicly validated via the required VM endpoint with 300,142 vehicles.